

A

PROJECT REPORT ON

“PIXELFORGE — BROWSER-BASED IMAGE EDITORP”

SUBMITTED TO:



**GOVT. TOOL ROOM AND TRAINING CENTRE
BELAGAVI**

In partial fulfillment for the award of

“DIPLOMA IN ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING”

BY

MOHAMMAD NIYAZ BAGAWAN

REG NO: 8080121

SHREYAS NILAJKAR

REG NO: 8080136

SOMARAJ A

REG NO: 8080155

DAYANANDA

REG NO: 8080152



CERTIFICATE

This is to certify that **Mr. MOHAMMAD NIYAZ BAGAWAN, Mr. SHREYAS NILAJKAR, Mr. SOMARAJ A and Mr. DAYANANDA** students of **Government Tool Room and Training Centre (GTTC), Belagavi, AIML Department**, have successfully completed the **Web Technology Lab Internal Assessment-II** project titled **“PixelForge — Browser-Based Image Processing and Editing Tool”** as part of the requirements for the **Diploma in Artificial Intelligence and Machine Learning**.

During the project, they demonstrated strong technical knowledge, sincerity, and dedication. The work involved designing and implementing a full-stack web application for real-time, client-side image processing. They successfully built an interactive browser-based image editor featuring **ten pixel-level editing tools (including OpenCV.js-powered morphological operations and contour-based shape detection)**, a **Web Worker-based processing pipeline**, a **multi-step undo/redo history system**, **live camera capture**, and **multi-format export (JPG, PNG, BMP)**. The application was developed using **React 19, Vite 8, Tailwind CSS**, and a **Python Flask backend skeleton**.

We wish them every success in their future academic and professional endeavors.

GUIDE
(MR.AMBAPRASAD)

H.O.D
(MR. RAMAKANTH)

PRINCIPAL
(MR. YALAPPA)

TABLE OF CONTENTS

Chapter	Content
Chapter 1	Introduction to PixelForge
Chapter 2	System Architecture and Technology Stack
Chapter 3	Modules and Features
Chapter 4	Implementation Details
Chapter 5	Workflow and User Journey
Chapter 6	Results and Conclusion

CHAPTER 1: INTRODUCTION

1.1 About PixelForge

PixelForge is a modern, fully browser-based image processing and editing web application developed as a capstone project for the Web Technology and AI practical lab. It is designed to simulate the capabilities of professional desktop image editors — entirely within the user's web browser, with no server-side image processing required.

The application is built on React 19, Vite 8, and Tailwind CSS for the frontend, and uses a Python Flask skeleton for the backend API layer. All image pixel computations are executed inside a dedicated Web Worker thread to ensure that the main UI thread remains fully responsive even while processing large high-resolution photographs.

PixelForge demonstrates core concepts in full-stack web application development, functional component architecture, client-side image manipulation using the HTML5 Canvas API, multi-threaded browser programming, and modern UI/UX design principles.

1.2 Project Objectives

The main objectives of developing PixelForge are:

- To build a fully functional, client-side image editor using React 19 and the HTML5 Canvas API.
- To implement real-time pixel-level image processing algorithms (grayscale conversion, brightness/contrast adjustment, box blur, convolution-based sharpening, Sobel edge detection, and RGB histogram computation).
- To leverage Web Workers for off-main-thread image processing, keeping the UI non-blocking and responsive.
- To provide dual image input modes — file upload with drag-and-drop, and live camera capture via the browser's `getUserMedia` API.
- To implement a complete undo/redo/reset history system using an immutable `ImageData` snapshot stack.
- To export edited images in JPG, PNG, and BMP formats, including a hand-encoded BMP encoder since browsers do not natively support BMP export.
- To apply light/dark theming using CSS custom properties, persisted across sessions via `localStorage`.
- To gain practical experience in component-based UI architecture, asynchronous JavaScript, and production build tooling.

1.3 Scope of the Project

PixelForge covers the following scope in its current beta release:

- A complete end-to-end editing workflow: Landing Page → Image Source Selection → Canvas Editor → Preview → Export.
- Ten independent editing tools accessible from a persistent left toolbar, including two OpenCV.js-powered tools (Morph and Detect).
- Full undo, redo, and reset support with a labeled history log.
- Support for JPG, PNG, BMP, WebP, and GIF as input formats.
- Export to JPG, PNG, and BMP output formats.
- Light and dark theme with OS preference detection and manual toggle.
- Responsive layout suitable for desktop and tablet use.

Project objectives — PixelForge

01 Client-side editor

React 19 + HTML5 Canvas API

02 Pixel algorithms

Blur, sharpen, edges, histogram

03 Web Worker pipeline

Off-thread, non-blocking UI

04 Dual image input

Upload + live camera capture

05 Undo / Redo history

Immutable ImageData stack

06 Multi-format export

JPG, PNG, hand-encoded BMP

07 Light / dark theming

CSS custom props + localStorage

08 Practical web skills

Async JS, components, bundling

☐ Core engineering

☐ Processing

☐ Data management

☐ UI / Experience

CHAPTER 2: SYSTEM ARCHITECTURE AND TECHNOLOGY STACK

2.1 System Architecture Overview

PixelForge follows a client-heavy Single Page Application (SPA) architecture. All image processing logic runs entirely within the browser. The backend Flask server is present as a skeleton for future API extension but is not required for any of the current editing features.

The application is divided into three logical layers:

- Presentation Layer — React functional components with Tailwind CSS utility classes and CSS custom property theming.
- Application Logic Layer — React state management in App.jsx using useState, useCallback, and useRef hooks, coordinating all pages, tools, history, and processing state.
- Image Processing Layer — Pixel operations implemented in imageProcessing.js, executed inside a dedicated Web Worker thread (imageWorker.js) to avoid UI blocking. Communication with the worker is managed by workerClient.js using a promise-based message-passing API.

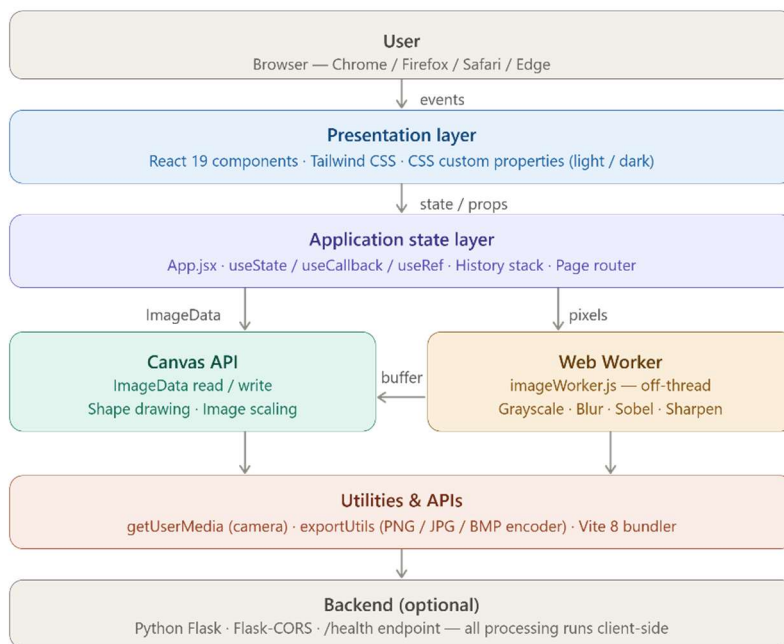
2.2 Project Folder Structure

DIRECTORY LAYOUT

pixelforge/	Root project directory
frontend/	React + Vite + Tailwind frontend application
src/components/	All UI components (Navbar, Toolbar, Panels, Modals)
src/pages/	Page-level components (Landing)
src/context/	React context providers (ThemeContext)
src/utils/	Image processing algorithms and export utilities
src/workers/	Web Worker for off-thread image computation
index.html	Vite HTML entry point
vite.config.js	Vite build configuration
tailwind.config.js	Tailwind CSS configuration
package.json	NPM dependencies and scripts
backend/	Python Flask backend skeleton
app.py	Flask app with /health endpoint

2.3 Technology Stack

Technology	Version	Role
React	19.2.8	UI component library and state management
Vite	8.2.0	Development server and production bundler
Tailwind CSS	3.4.19	Utility-first CSS framework for styling
JavaScript ES2022	—	Core language; Web Workers, Canvas API, Fetch
HTML5 Canvas API	—	Pixel-level image read/write and shape drawing
Web Workers API	—	Off-main-thread image computation
getUserMedia API	—	Live camera capture in the browser
Python Flask	—	Lightweight backend HTTP server skeleton
Flask-CORS	—	Cross-origin resource sharing middleware
OxLint	1.75.0	Fast Rust-based JavaScript linter



CHAPTER 3: MODULES AND FEATURES

3.1 Landing Page Module

The Landing page (`Landing.jsx`) is the first screen the user encounters. It features an animated hero section with floating gradient background orbs, a PixelForge brand logo, a tagline ('Reforge your images pixel by pixel.'), and a prominent 'Start Editing' call-to-action button that transitions the app to the Image Source selection screen.

The landing page also includes a feature highlights row listing the three core value propositions: pixel-level editing tools, upload or live camera capture, and one-click multi-format export. A light/dark theme toggle button is visible in the navigation bar.

3.2 Image Source Selection Module

The `ImageSourceScreen` component presents the user with exactly two image input options:

1. **Upload Image** — A drag-and-drop file picker zone that accepts image files of type JPG, PNG, BMP, WebP, and GIF. The user can either click to open the system file browser or drag an image file directly onto the drop zone. File type validation is performed client-side before loading.
2. **Take Image (Camera Capture)** — The `CameraCapture` component activates the device camera using the browser's `navigator.mediaDevices.getUserMedia` API. The user sees a live video preview with controls to flip between front and rear cameras, capture a still frame, retake the photo, or cancel. The captured frame is converted to a `Blob` and passed through the same image loading pipeline as an uploaded file.

3.3 Image Loading and Scaling Pipeline

When an image is loaded (either via upload or camera capture), `App.jsx` runs it through a standardized loading pipeline:

1. `createImageBitmap()` is used to decode the image off the main thread, using the browser's native (often hardware-accelerated) image decoder.
2. The image is inspected for its natural dimensions. If the longer edge exceeds 1,600 pixels, the image is downscaled proportionally to fit within this limit. This hard cap ensures that subsequent pixel-by-pixel editing operations remain fast regardless of the source camera resolution.
3. The decoded bitmap is drawn onto an off-screen `<canvas>` element at the target dimensions using `drawImage`.
4. `getImageData()` extracts the raw RGBA pixel buffer as an `ImageData` object, which is stored as the initial history entry and becomes the working surface for all edits.

3.4 Canvas Editor — Image Workspace

The ImageWorkspace component renders the current ImageData to a visible <canvas> element in the editor viewport. It also handles the interactive Shapes tool:

- When the Shapes tool is active, the canvas becomes interactive for pointer events.
- The user clicks and drags to define the start and end points of a shape.
- A live preview of the shape is drawn on every pointer-move event onto a temporary overlay canvas, providing real-time visual feedback without modifying the actual pixel history.
- On pointer-up, the shape is committed: the shape is drawn permanently onto a copy of the current ImageData and pushed as a new history entry via onCommitShape.
- Supported shapes include Rectangle, Circle (ellipse), and Line, with configurable stroke colour, fill colour, fill toggle, and stroke width.

3.5 Left Toolbar — Tool Selection

The LeftToolbar component displays a vertical strip of eight icon buttons, one for each editing tool. The active tool is highlighted with an accent-coloured background. While a processing operation is in progress, all toolbar buttons are disabled with a visual wait cursor to prevent concurrent edits. The available tools are:

Tool ID	Icon	Function
upload	↑	Re-open image source screen to load a different image
grayscale	⦿	Convert image to luminance-weighted grayscale
brightness	☀	Adjust brightness and contrast with two sliders
blur	○	Apply a fast separable box blur with radius control
sharpen	◆	Apply convolution-based unsharp mask sharpening
edges	⬡	Sobel edge detection with threshold control
histogram	▨	Display live RGB and luminance histogram chart
shapes	△	Draw rectangle, circle, or line directly on canvas
morph	⬢	Morphological operations: dilate, erode, open, close (via OpenCV.js WebAssembly)
detect	⬡	Contour-based shape detection using OpenCV findContours + approxPolyDP; classifies triangles, squares, rectangles, pentagons, hexagons, and circles

3.6 Right Panel — Tool Settings and History

The RightPanel component renders context-sensitive controls for the currently active tool. For tools with parameters (brightness, contrast, blur radius, sharpen amount, edge threshold), it displays labelled range sliders with real-time value readouts. Slider events are throttled to one React state update per animation frame using `requestAnimationFrame` to keep drag interactions smooth.

The panel also contains:

- Apply button — Sends the current tool operation and parameters to the Web Worker via the `applyEdit` function in `App.jsx`.
- Undo / Redo / Reset buttons — Traverse the history stack or restore the original loaded image.
- History log — A scrollable list of labeled history entries (e.g., 'Applied Blur (r=4)', 'Adjusted Brightness/Contrast') showing the complete edit chain.
- Export section — Quick download buttons for JPG, PNG, and BMP formats, plus a 'Preview Result' button that opens the Preview Modal.

3.7 Morphology Tool Module (Morph)

The Morph tool provides four OpenCV.js-powered morphological operations directly on the pixel buffer: Dilation, Erosion, Opening, and Closing. Morphological operations are fundamental computer-vision primitives that grow or shrink bright regions in an image based on a structurally-shaped kernel.

Dilation expands bright regions, useful for connecting broken edges or making features more prominent. Erosion shrinks bright regions, removing small noise speckles. Opening (erode then dilate) is the standard noise-removal preset that eliminates small foreground blobs without altering the overall shape of larger objects. Closing (dilate then erode) fills small holes and gaps in foreground regions.

All four operations are backed by real OpenCV calls — `cv.dilate`, `cv.erode`, and `cv.morphologyEx` — executing inside a dedicated `opencvWorker.js` Web Worker thread so the main UI thread is never blocked during WASM computation. The user can adjust the kernel size (3, 5, 7, or 9 pixels) and the number of iterations (1 to 5) using sliders in the Right Panel. OpenCV.js is lazy-loaded on demand, with a background warm-up fetch beginning as soon as the editor opens.

3.8 Shape Detection Tool Module (Detect)

The Detect tool uses OpenCV contour analysis to automatically find and classify geometric shapes in the current image. It is implemented using `cv.findContours` and `cv.approxPolyDP`, which approximate each detected contour as a polygon. The number of vertices of the approximated polygon determines the shape classification: 3 vertices → Triangle, 4 vertices → Square or Rectangle (aspect-ratio discriminated), 5 → Pentagon, 6 → Hexagon, and higher vertex counts or near-circular contours → Circle or general Polygon.

Before contour extraction, a noise-filtering pre-pass is applied: the image is converted to grayscale, binarised using Otsu's automatic threshold, and cleaned with a morphological open-close pass. This significantly reduces false positives on textured or noisy images. Detected shapes are outlined and labelled directly on the canvas, and a summary list of all detected shape types and counts is displayed in the Right Panel. A sensitivity threshold slider allows the user to control the polygon approximation epsilon parameter, trading off detection sensitivity against false positive rate.

3.9 Preview Modal

The PreviewModal component renders the current ImageData to a preview canvas inside a full-screen overlay modal. It displays the image at its actual edited dimensions and provides three one-click download buttons corresponding to the three supported export formats. The modal can be dismissed by clicking the close button or clicking outside the modal content area.

3.10 Theme System

The ThemeContext React context wraps the entire application and manages the active colour theme (light or dark). The theme preference is read from localStorage on load, falling back to the user's OS-level colour scheme preference via the prefers-color-scheme media query. The ThemeToggle component exposes a sun/moon icon button that toggles and persists the selected theme. All colours throughout the application are defined as CSS custom properties (--forge-bg, --forge-surface, --forge-accent, etc.) in index.css, so the entire visual appearance updates instantly when the theme class is toggled on the document root without any component re-renders.

CHAPTER 4: IMPLEMENTATION DETAILS

4.1 Image Processing Algorithms

All pixel-level operations are implemented as pure functions in `imageProcessing.js`. Each function receives an `ImageData` object, creates a deep clone of the underlying `Uint8ClampedArray` using `new Uint8ClampedArray(imageData.data)`, performs the transformation on the clone, and returns a new `ImageData` — never mutating the input. This immutability guarantee is fundamental to the correctness of the undo history stack.

4.1.1 Grayscale Conversion

Luminance-weighted grayscale is computed using the ITU-R BT.601 coefficients: $Y = 0.299 \cdot R + 0.587 \cdot G + 0.114 \cdot B$. This produces perceptually accurate grey tones that match human eye sensitivity. The computed luminance value is written back to all three channels (R, G, B) of each pixel, leaving the alpha channel unchanged.

4.1.2 Brightness and Contrast Adjustment

Brightness adjustment adds a flat offset $b = (\text{brightness} / 100) \times 255$ to each channel. Contrast adjustment uses the standard linear contrast formula with scaling factor $f = (259 \times (c + 255)) / (255 \times (259 - c))$, applied as: $\text{channel} = \text{clamp}(f \times (\text{channel} - 128) + 128 + b)$. The clamp function keeps all values within the valid $[0, 255]$ range. Parameters are independently adjustable on sliders ranging from -100 to $+100$.

4.1.3 Box Blur

A separable box blur is applied using a two-pass sliding window algorithm (horizontal pass followed by a vertical pass). To approximate a Gaussian blur, three passes of the box blur are applied sequentially. The sliding window approach runs in $O(W \times H)$ time per pass regardless of the blur radius, making large-radius blurs computationally affordable. Edge pixels use clamped boundary conditions.

4.1.4 Unsharp Mask (Sharpen)

Sharpening is implemented as a discrete 3×3 convolution using the kernel: $[0, -a, 0, -a, 1+4a, -a, 0, -a, 0]$, where a is the user-controlled amount parameter (range 0–5). This is the standard unsharp mask laplacian kernel; higher values of a produce progressively stronger edge enhancement.

4.1.5 Sobel Edge Detection

Edge detection first converts the image to grayscale, then applies two separate 3×3 Sobel convolution kernels — G_x for horizontal gradients and G_y for vertical gradients. The gradient magnitude $M = \sqrt{G_x^2 + G_y^2}$ is computed for each pixel. An optional threshold parameter suppresses weak edges by setting pixels below the threshold to zero, making the output cleaner for high-noise images.

4.1.6 RGB Histogram

The histogram function iterates through all pixels and counts the frequency of each value (0–255) in the Red, Green, Blue, and luminance channels independently, producing four arrays of 256 counters. The resulting histogram data is rendered as a canvas-drawn overlay chart in the RightPanel's HistogramChart sub-component, with separate coloured curves for R, G, and B.

4.1.7 Morphological Operations (OpenCV.js)

Morphological operations are implemented in `opencvProcessing.js` using the OpenCV.js WebAssembly build. Each operation converts the source `ImageData` into an OpenCV Mat object (`cv.matFromImageData`), constructs a structuring element kernel of the requested size using `cv.getStructuringElement` with a rectangular kernel shape, then applies the selected operation via `cv.dilate`, `cv.erode`, or `cv.morphologyEx` (with `cv.MORPH_OPEN` or `cv.MORPH_CLOSE` as the operation flag). The result Mat is read back into a new `ImageData` using `cv.imencode`-equivalent pixel copy, and all intermediate Mat objects are explicitly deleted to avoid WASM memory leaks.

These operations execute inside `opencvWorker.js`, a dedicated Web Worker thread that loads the OpenCV.js WASM binary once on first use. Communication with the worker follows the same promise-based message-passing pattern used for the JS image worker, with Transferable pixel buffers for zero-copy data transfer between the main thread and the worker.

4.1.8 Contour-Based Shape Detection (OpenCV.js)

Shape detection is implemented in `opencvProcessing.js` using the following pipeline: (1) Convert to grayscale using `cv.cvtColor(src, gray, cv.COLOR_RGBA2GRAY)`. (2) Apply Otsu automatic thresholding via `cv.threshold` with `cv.THRESH_OTSU` to produce a binary image. (3) Apply a morphological open then close pass to remove noise. (4) Find contours using `cv.findContours` with `cv.RETR_EXTERNAL` and `cv.CHAIN_APPROX_SIMPLE`. (5) For each contour, approximate it to a polygon using `cv.approxPolyDP` with `epsilon = sensitivity * cv.arcLength(contour, true)`. (6) Classify by vertex count and aspect ratio. (7) Draw the outline and label using `cv.polyline` and `cv.putText` on the output Mat. All intermediate Mat and MatVector objects are explicitly freed after use.

4.2 Web Worker Architecture

To prevent image processing from freezing the browser's main thread (which handles all UI rendering, event handling, and React reconciliation), all computationally expensive pixel operations are executed inside a dedicated Web Worker thread (`imageWorker.js`).

The `workerClient.js` module implements a promise-based wrapper around the Web Worker's message-passing API. Each call to `runImageOp()` generates a unique integer ID, sends a message to the worker with the operation name, image dimensions, parameters, and a Transferable pixel buffer (using zero-copy buffer transfer to avoid the overhead of copying megabytes of pixel data), and registers a resolution

callback. When the worker responds with the matching ID, the promise resolves with the result `ImageData`. If the worker throws an error, the promise rejects.

The worker is pre-warmed via `warmupWorker()` on application mount so that the thread startup cost (parsing, JIT compilation) is absorbed during idle time before the user's first Apply click.

4.3 Export System

The `exportUtils.js` module handles three export formats:

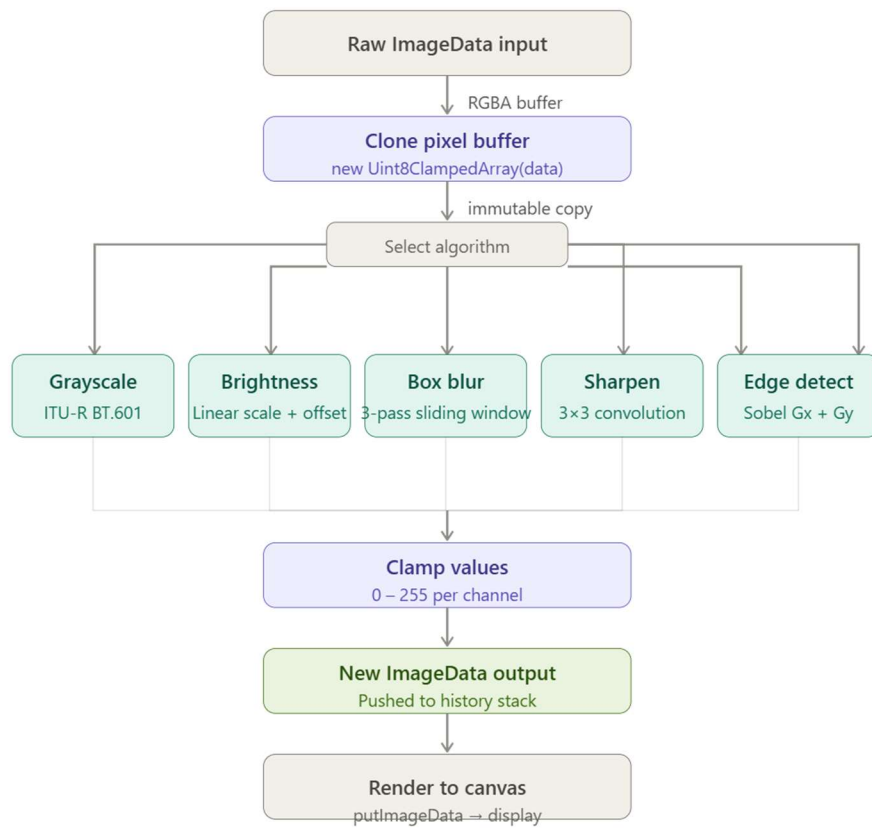
- PNG — `canvas.toBlob(callback, 'image/png')` produces lossless PNG output.
- JPG — Before calling `toBlob`, the `ImageData` is composited over a white background rectangle (`ctx.fillStyle = 'ffffff'`) to flatten the alpha channel, preventing the black-background artifact that occurs when JPEG (which has no alpha channel) tries to represent transparency.
- BMP — Since no major browser's `canvas.toBlob()` implementation supports `'image/bmp'` reliably, a complete uncompressed 24-bit BMP file is hand-encoded directly from the `ImageData` pixel buffer. The encoder writes a `BITMAPFILEHEADER` (14 bytes) and a `BITMAPINFOHEADER` (40 bytes), then writes the pixel data in BMP's required bottom-up row order with BGR channel order and 4-byte row padding. The resulting `ArrayBuffer` is wrapped in a `Blob` for download.

4.4 History and State Management

The undo/redo system is implemented as a linear history stack in `App.jsx`. The state consists of two values: `history` (an array of `{ label, data: ImageData }` entries) and `historyIndex` (the integer index of the currently active entry).

- Undo — decrements `historyIndex`, restoring the previous `ImageData` without deleting future entries.
- Redo — increments `historyIndex`, reapplying a previously undone edit.
- New edit — truncates the history array to `historyIndex + 1` (discarding any redo-able entries) and appends the new entry, then increments `historyIndex`.
- Reset — truncates history to only the first entry (the originally loaded image) and sets `historyIndex` to 0.

A concurrent processing guard using a React `useRef` flag (`isProcessingRef`) prevents race conditions when the user rapidly clicks Apply: the ref is set synchronously before the async worker call and cleared in the finally block, ensuring that two operations can never be in flight simultaneously even if React has not yet committed the `isProcessing` state update.



CHAPTER 5: WORKFLOW AND USER JOURNEY

5.1 Complete Application Workflow

PixelForge is designed around a single, linear editing workflow that guides the user from image selection through to export. The workflow consists of the following stages:

Stage 1 — Landing Page

The user arrives at the animated landing page and reads the product tagline and feature summary. Clicking 'Start Editing' transitions the application to the Source Selection screen.

Stage 2 — Image Source Selection

The user chooses between 'Upload Image' (file picker / drag-and-drop) and 'Take Image' (live camera). The selected or captured image is validated, decoded, optionally downscaled to a maximum working dimension of 1,600 pixels on the longest edge, and loaded into the editor as the initial history entry.

Stage 3 — Editing

The full editor layout becomes visible, consisting of the Navbar at the top, the Left Toolbar on the left, the Image Workspace canvas in the centre, and the Right Panel on the right. The user selects a tool from the Left Toolbar; the Right Panel immediately updates to display the controls relevant to that tool. After adjusting parameters using sliders, the user clicks 'Apply' to dispatch the operation to the Web Worker. The processed ImageData is pushed to the history stack and the canvas updates to display the new result. This process can be repeated across multiple tools in any order.

Stage 4 — Undo / Redo / Reset

At any point, the user can undo the last applied operation, redo an undone operation, or reset the image to its originally loaded state using the history controls in the Right Panel. The labeled history log in the Right Panel shows the complete sequence of operations applied.

Stage 5 — Preview and Export

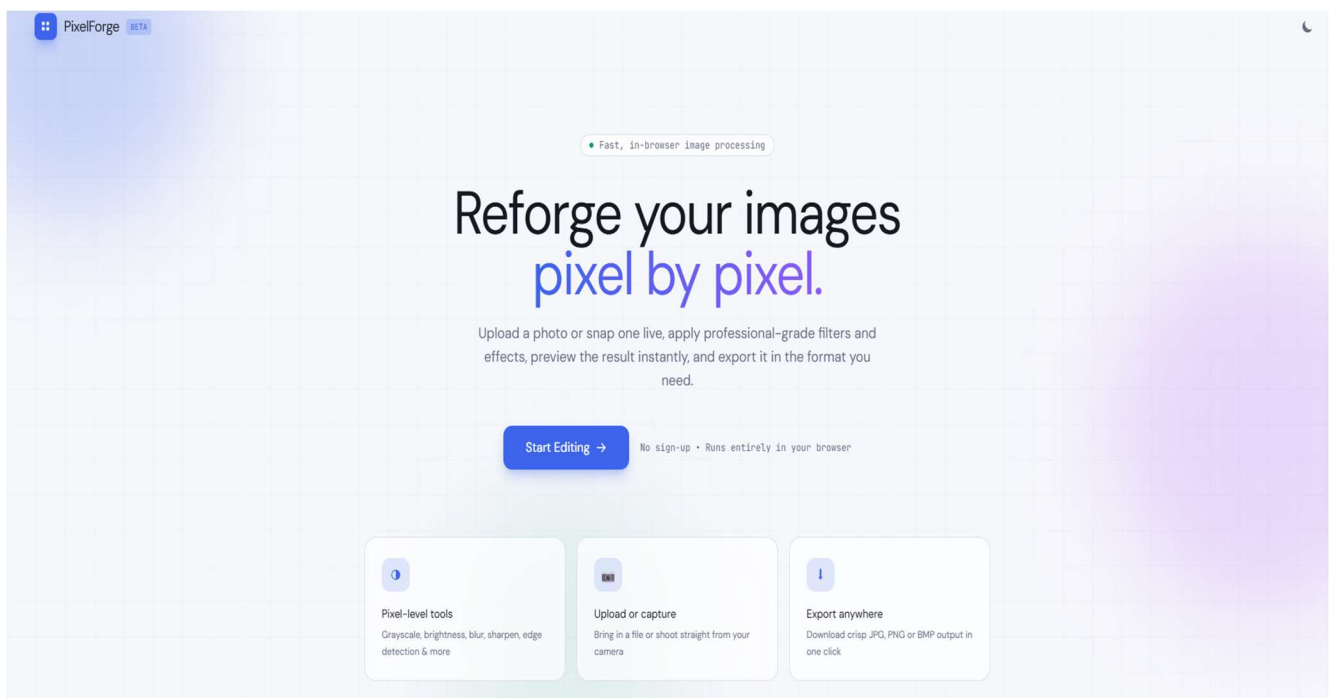
The user clicks 'Preview Result' to open the Preview Modal, which renders the final edited image at full resolution. From the modal (or directly from the Right Panel), the user clicks the desired format button (JPG, PNG, or BMP) to trigger a client-side download of the edited image. No data is sent to any server during this step.

5.2 Application State Machine

The application navigates between three named pages managed by a single page state variable in App.jsx:

Page State	Screen Shown
'landing'	Animated landing page with 'Start Editing' CTA
'source'	Image source selection (Upload / Camera)
'editor'	Full editor with Toolbar, Canvas, Right Panel

Figure 3.1: PixelForge — Application Dashboard



CHAPTER 6: RESULTS AND CONCLUSION

6.1 Project Outcomes

PixelForge was successfully developed and tested as a fully functional browser-based image editing application. All ten planned editing tools were implemented with real, pixel-level processing algorithms — not CSS filters or pre-built library calls. The following functional milestones were achieved:

1. A complete end-to-end editing workflow was implemented covering landing, image input, editing, preview, and multi-format export.
2. All eight image processing operations (grayscale, brightness/contrast, blur, sharpen, edge detection, histogram, and shapes) are computationally real and operate directly on the pixel buffer.
3. The Web Worker architecture ensures the application UI remains responsive even while processing high-resolution images.
4. Both file upload and live camera capture input methods are fully operational.
5. The BMP hand-encoder correctly produces valid, non-empty BMP files without relying on unavailable browser APIs.
6. The undo/redo/reset history system maintains full edit history with labeled entries.
8. Morphological image operations (dilation, erosion, opening, closing) are fully implemented using OpenCV.js (WebAssembly), loaded on demand with a background warm-up fetch so the binary is ready before the user selects either tool.
9. Contour-based shape detection correctly classifies triangles, squares, rectangles, pentagons, hexagons, and circles using OpenCV findContours and approxPolyDP, with Otsu thresholding and morphological noise-filtering applied before contour extraction.
10. The histogram now correctly distinguishes true grayscale images (single intensity curve) from colour images (separate R, G, B curves). Camera capture was also optimised to perform a single encode pass, matching the same resolution cap as file uploads.
7. Light and dark themes are applied instantly using CSS custom properties, with persistence across browser sessions.

6.2 Technical Achievements

The project demonstrates several advanced web development techniques that go beyond standard tutorial-level implementations:

- Zero-copy Transferable buffer passing between the main thread and the Web Worker, eliminating memory duplication overhead for large images.
- A race-condition-safe processing lock using useRef for synchronous state, avoiding a known limitation of React's asynchronous setState.
- A three-pass separable box blur as a Gaussian approximation, running in linear $O(W \times H)$ time independently of the blur radius.

- A complete BMP file encoder built from the BITMAPFILEHEADER and BITMAPINFOHEADER binary specifications, handling bottom-up row order, BGR colour ordering, and 4-byte row padding.
- An image downscaling step on load (capped at 1,600px) that uses createImageBitmap for hardware-accelerated decoding, with a fallback to an element for edge-case formats.

Lazy-loading of OpenCV.js (WASM) with a background warm-up fetch initiated at editor mount, ensuring the binary is parsed and ready before the user selects a morphology or detection tool, eliminating noticeable startup latency on first use.

A dedicated opencvWorker.js thread that executes all OpenCV WASM operations (cv.dilate, cv.erode, cv.morphologyEx, cv.findContours, cv.approxPolyDP) off the main thread, keeping the application UI fully responsive during compute-intensive computer-vision operations.

An Otsu automatic threshold combined with a morphological open-close noise-filtering pre-pass applied before contour extraction in the shape detection pipeline, significantly reducing false positives on textured or photographically complex images without requiring any manual tuning beyond the single sensitivity slider.

6.3 Conclusion

PixelForge successfully demonstrates the capability of modern web technologies to deliver professional-grade image processing functionality entirely within the browser, with no dependency on server-side computation for its core features. The project provided hands-on experience with React 19 component architecture, the HTML5 Canvas API, Web Workers for multi-threaded browser programming, the getUserMedia camera API, binary file encoding, and client-side state management at scale.

The skills and design patterns employed in PixelForge — immutable data structures, worker-based concurrency, Transferable objects, pure processing functions, and CSS custom property theming — are directly applicable to production-grade applications in the fields of AI-assisted image analysis, computer vision tooling, and Industry 4.0 smart manufacturing systems.

6.4 Future Enhancements

The following enhancements are planned for future iterations of PixelForge:

- Integration of the Flask backend API to support server-side AI-based image enhancements (super-resolution, background removal, object detection).
- Addition of image cropping, rotation, and free-transform tools.
- Support for saving and loading edit sessions as JSON project files.
- Progressive Web App (PWA) support for offline use and installability.
- Batch processing mode for applying the same edit pipeline to multiple uploaded images.

- WebAssembly (WASM) acceleration for additional compute-intensive operations such as large-radius Gaussian blur and multi-scale edge detection (OpenCV.js WASM is already integrated for morphological operations and shape detection in this release).

6.5 References

1. React Documentation — react.dev
2. Vite Build Tool Documentation — vitejs.dev
3. Tailwind CSS Documentation — tailwindcss.com
4. MDN Web Docs: Canvas API — developer.mozilla.org/en-US/docs/Web/API/Canvas_API
5. MDN Web Docs: Web Workers API — developer.mozilla.org/en-US/docs/Web/API/Web_Workers_API
6. MDN Web Docs: MediaDevices.getUserMedia — developer.mozilla.org/en-US/docs/Web/API/MediaDevices/getUserMedia
7. Microsoft BMP File Format Specification — docs.microsoft.com
8. ITU-R BT.601 Luma Coefficients — itu.int
9. Flask Documentation — flask.palletsprojects.com